

SYSTEM DESIGN DOCUMENT

ToolNest & Portfolio Architecture, Patterns & Engineering Decisions

An end-to-end technical reference for the two applications hosted at **shashikanth077.github.io** — a Next.js portfolio and ToolNest, a Module-Federation-based, 38-tool, browser-only utilities platform.

Author

Shashikanth Hosur Ramegowda

Repository

shashikanth077.github.io

Generated

August 1, 2026

Next.js 12

React 19

Vite 7

Module Federation

TypeScript 5.7 strict

Redux Toolkit

§0 Table of Contents

1. Executive Summary	03
2. System Overview & Deployment Topology	03
3. Portfolio — Architecture Deep Dive	04
4. ToolNest — Monorepo & Workspace Structure	05
5. Module Federation Runtime Architecture	06
6. State Management Strategy	07
7. Design System & Theming	08
8. Tool Catalog (38 Tools / 5 Toolkits)	09
9. Key Libraries & Why They Were Chosen	10
10. Cross-Cutting Design Patterns	11
11. Performance Engineering — The Preboot Loader	13
12. CI/CD & Deployment Pipeline	14
13. Skills Demonstrated — Reviewer's Summary	15

§1 Executive Summary

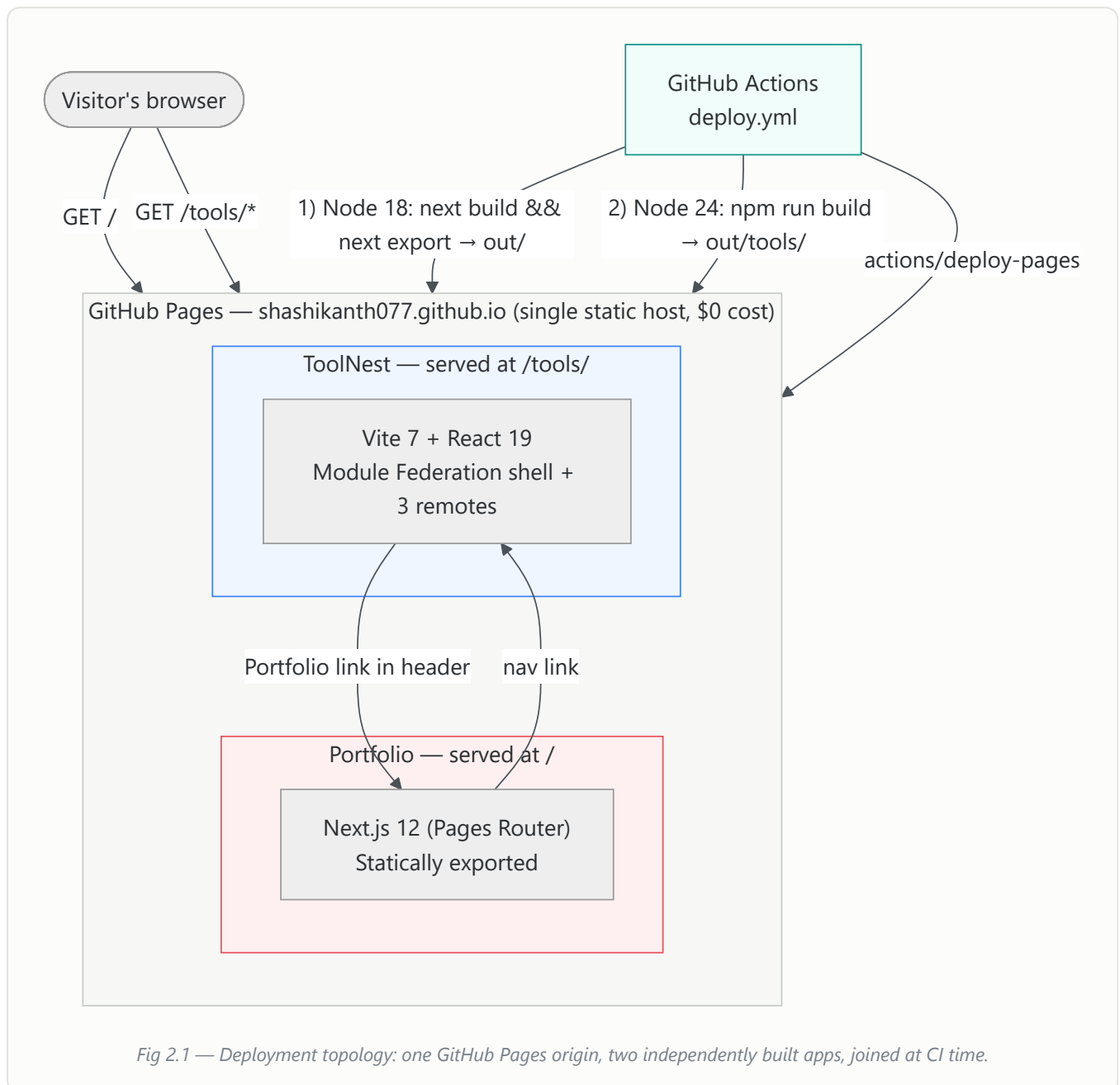
This repository hosts two independently built, independently deployed applications on a single GitHub Pages origin at zero infrastructure cost — a personal portfolio and ToolNest, a privacy-first, browser-only toolkit platform. Despite sharing a domain, they share almost nothing in code: different frameworks, different Node versions, different build tools, joined only at deploy time by a single GitHub Actions pipeline that assembles both into one static `out/` directory.



The document that follows walks the system end to end: how the two apps are deployed together, how ToolNest's micro-frontend architecture is wired, what state lives where and why, the design token system that drives theming across five color-coded toolkits, and the specific engineering patterns — error containment, shared-dependency singletons, static prerendering for SPA SEO, and a JS-independent preboot loader — that make a browser-only, serverless tools platform behave like a production application rather than a demo.

§2 System Overview & Deployment Topology

Both applications are static output committed to a single GitHub Pages origin. The portfolio owns the root path `/`; ToolNest owns `/tools/`. Neither app is aware of the other's internals — the only coupling is a nav link each direction and a shared CI pipeline that must run the portfolio's export *before* the tools build, because `next export` rewrites the `out/` directory wholesale and would silently delete anything already written there.



Why this split matters: it demonstrates that "microservices thinking" isn't limited to backends. Two frontends with completely different tech stacks, release cadences, and even Node.js versions can share one deployment

target without coupling their build systems together — the same discipline ToolNest itself applies one level deeper, between its shell and remotes.

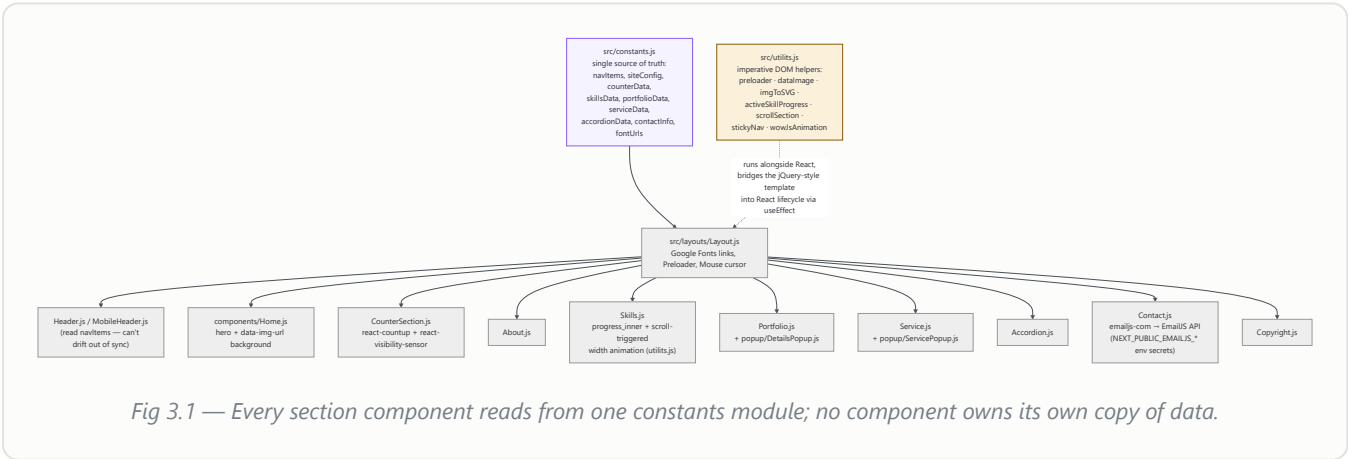
§3 Portfolio — Architecture Deep Dive

A Next.js 12 (Pages Router) single-page marketing site, statically exported for GitHub Pages, built around a "define once, import everywhere" data model.

Tech Stack

LAYER	CHOICE	NOTES
Framework	Next.js 12.3.4 (Pages Router)	Static export mode (<code>next export</code>) — no server runtime, pure static HTML/CSS/JS for GitHub Pages
UI Library	React 18.2	Class-free, hooks-based components throughout
Styling	Global CSS (plugins.css, dark.css, style.css) + CSS custom properties	Template-derived (Marketify), adapted rather than rewritten from scratch
Animation	wowjs, react-countup, react-visibility-sensor	Scroll-triggered reveal + animated counters on the About section
Carousel	swiper 8.3	Testimonials / partner logos
Contact form	emailjs-com	Client-side email send with no backend — service/template/public keys injected via <code>NEXT_PUBLIC_*</code> env vars at CI build time

Component Architecture



The single-source-of-truth pattern

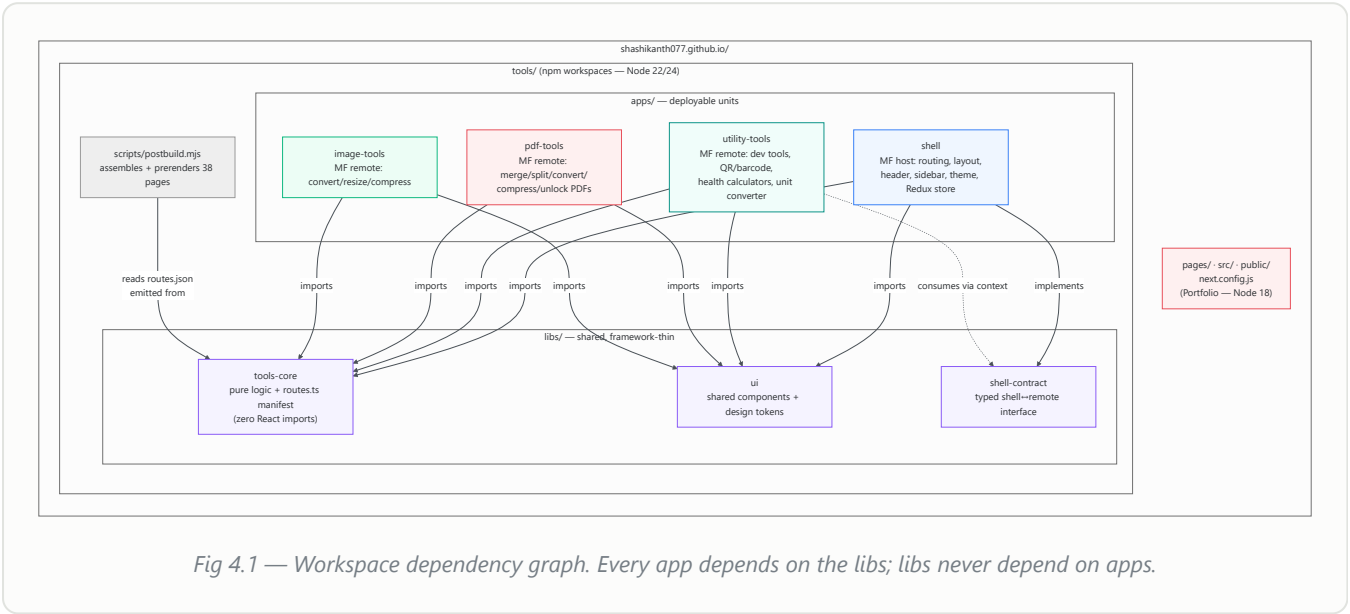
`src/constants.js` is the one file that owns every piece of copy, every link, and every data array on the site — navigation items, social links, skill percentages, portfolio project metadata, service descriptions, FAQ/accordion content, and even the Google Fonts URLs. Both the desktop `Header.js` and the separate `MobileHeader.js` import the same `navItems` array, which is what guarantees the two menus can never drift out of sync — a real risk in any template with duplicated markup for responsive breakpoints.

Bridging an imperative template into React

The visual template (Marketify) was originally built for jQuery-style imperative DOM manipulation — scroll-triggered skill bar fills, a custom mouse cursor, a WOW.js reveal-on-scroll library, sticky header state. Rather than rewriting all of that as idiomatic React state, `src/utlilits.js` exposes a set of plain functions (`activeSkillProgress`, `dataImage`, `stickyNav`, `customCursor`, etc.) that each component wires in via `useEffect` on mount. This is a deliberate compromise: it keeps the proven visual behavior of the template intact while still hosting everything inside React's component tree and Next.js's build pipeline.

§4 ToolNest — Monorepo & Workspace Structure

ToolNest is an npm-workspaces monorepo with a strict two-tier split: `apps/` are independently deployable units (a Module Federation host and three remotes); `libs/` are shared code with no deployment identity of their own.



Why three libs, not one

PACKAGE	RESPONSIBILITY	KEY CONSTRAINT
<code>@devtools/tools-core</code>	Pure business logic: PDF/image/QR/JWT/health-math functions, plus the <code>routes.ts</code> manifest	Zero React imports — <code>sideEffects: false</code> lets Rollup tree-shake unused feature modules (e.g. a JSON tool never pulls in the 1.2MB bwip-js barcode library)
<code>@devtools/ui</code>	Shared presentational components (Button, Panel, Note, Spinner, ToolFrame, CopyButton) and the CSS design-token system	Peer-depends on React 19 rather than bundling it, so it never creates a second React copy
<code>@devtools/shell-contract</code>	The <i>only</i> typed interface a remote may use to call back into the shell	Deliberately tiny — <code>navigate()</code> , <code>toast()</code> , <code>theme</code> — every addition here is a coupling point that all four apps depend on simultaneously

The route manifest as single source of truth

`libs/tools-core/src/routes.ts` is the ToolNest equivalent of the portfolio's `constants.js`: one array of `ToolRoute` objects (slug, name, tagline, SEO description, keywords, toolkit, category, owning remote, icon) that three independent consumers read from — the shell's router, the remote's lazy-loaded route table, and `scripts/postbuild.mjs`, which uses it to prerender 38 static HTML files. A route can be marked `hidden` to

keep it usable in local dev while excluding it from every production navigation surface and the sitemap — the mechanism used to soft-launch the in-progress `edit-pdf` tool.

§5 Module Federation Runtime Architecture

ToolNest's four deployable apps — `shell`, `utility-tools`, `pdf-tools`, `image-tools` — are built completely independently by Vite, then wired together at *runtime* via `@module-federation/vite`, not at build time. The shell never imports a remote's source code; it fetches the remote's compiled `remoteEntry.js` over HTTP the first time a route needs it.

Browser requests
/tools/bmi-calculator

Prerendered index.html
(per-route SEO metadata +
JSON-LD)

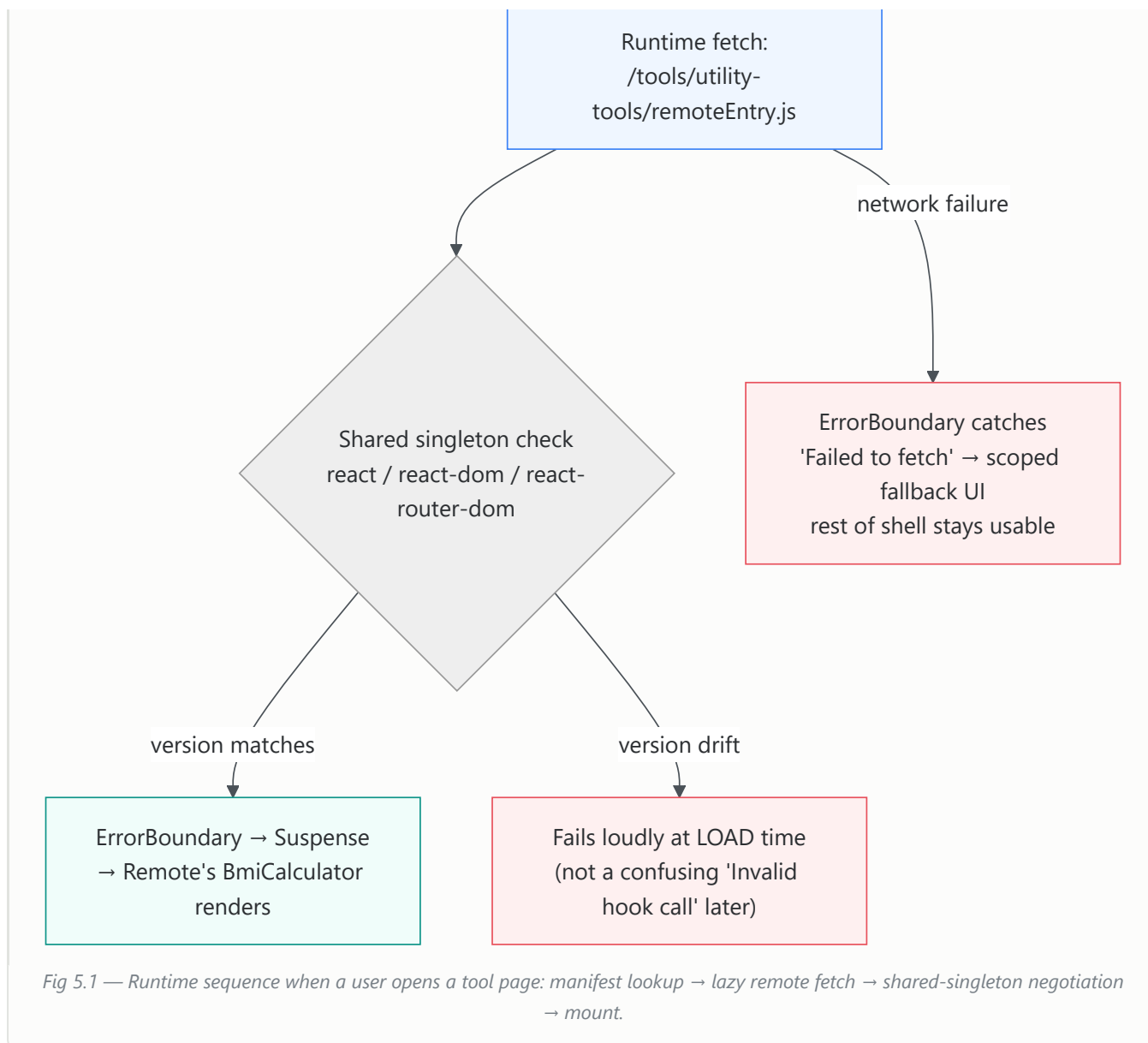
Shell boots: React 19 +
Redux store
+ react-router-dom v7 +
TanStack Query

Router matches :slug

findTool('bmi-calculator')
via routes.ts manifest

tool.remote === 'utility'

lazy(() =>
import('utility_tools/ToolRoutes'))



Shared dependency singletons

The riskiest part of any Module Federation setup is dependency drift: if the shell and a remote each bundle their own copy of React, hooks silently break with the notoriously unhelpful *"Invalid hook call"* error at render time. Every one of ToolNest's four Vite configs declares `react`, `react-dom`, and `react-router-dom` as `{ singleton: true, strictVersion: true }` shared modules. With that flag, a version mismatch fails loudly the moment the remote is *loaded* — a clear console error at a known point in time — rather than surfacing later as a confusing runtime crash deep inside a component.

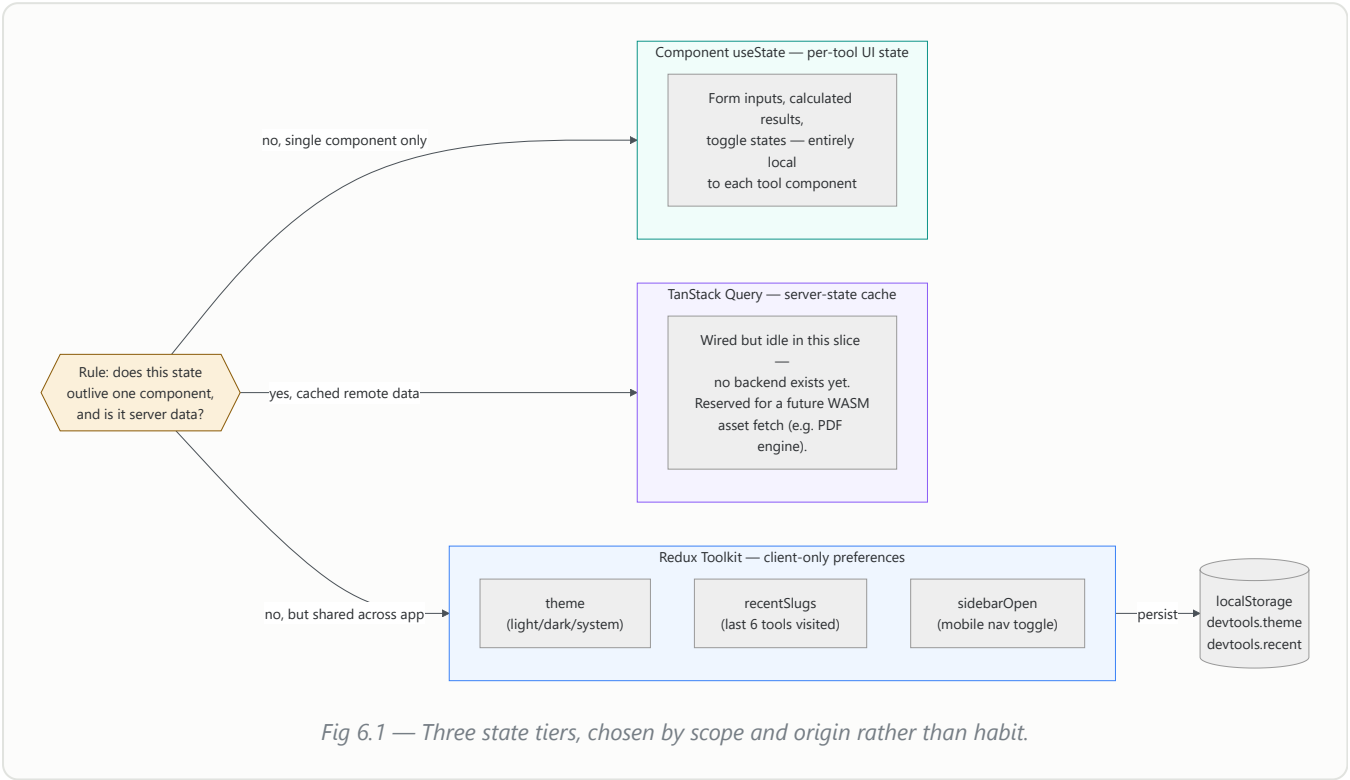
Containment via Error Boundaries

Because remotes are fetched independently at runtime, a network hiccup or a bad deploy of one remote must not blank the entire application. `ToolPage.tsx` wraps every lazily-loaded remote in a scoped `ErrorBoundary` (keyed by the route slug, so navigating away and back automatically clears a stuck error) and a `Suspense` boundary showing the shared `Spinner` component. The boundary distinguishes fetch/network failures from in-tool runtime errors and shows a tailored message either way — this is the

entire payoff of splitting the app into micro-frontends in the first place: partial, contained failure instead of a blank page.

§6 State Management Strategy

State is split across three tools by a single question: does it outlive one component, and is it server data? ToolNest has no backend at all in this slice, which makes the boundary unusually clean.

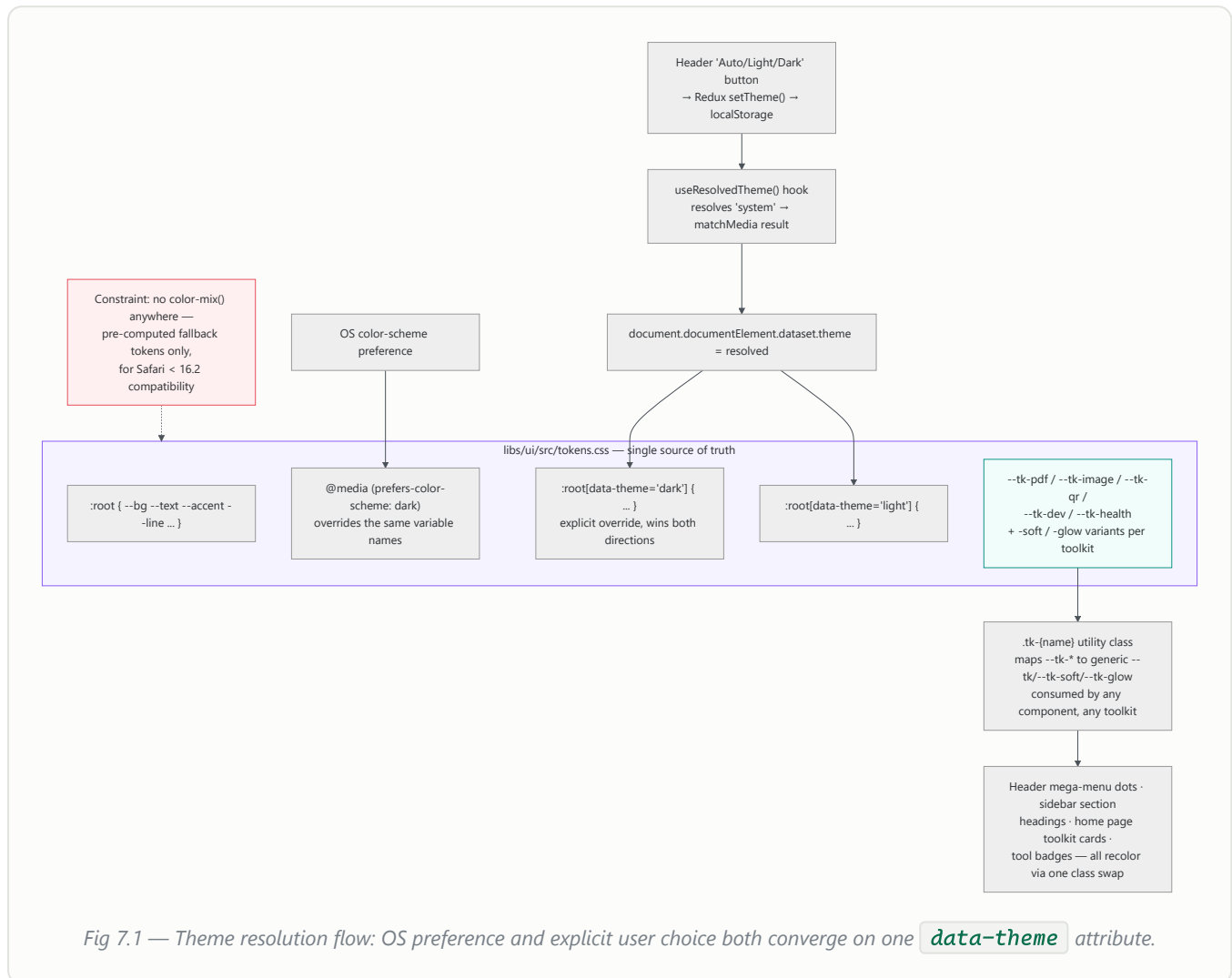


TOOL	HOLDS	PERSISTENCE
Redux Toolkit	Theme preference, last 6 visited tools, mobile sidebar open/closed	<code>localStorage</code> , read defensively (try/catch around private-mode or blocked storage)
TanStack Query	Nothing yet — configured and wired into the provider tree, but genuinely idle	N/A — reserved for the day a tool needs to fetch a remote asset, e.g. a WASM PDF engine
Component <code>useState</code>	Every calculator's form inputs and computed results, every converter's live output	None — intentionally ephemeral, resets on navigation

Engineering judgment on display: TanStack Query is included specifically because it is *not yet needed* — wiring the provider now means the day a tool needs cached remote data, no shell-level refactor is required. Documented candidly in code comments rather than presented as load-bearing infrastructure it isn't.

§7 Design System & Theming

One CSS custom-property file (`libs/ui/src/tokens.css`) is imported by both the shell and every remote's standalone dev entry point, so a remote still looks correct when run in isolation on its own Vite dev port.



Dual dark-mode strategy

Two mechanisms set the same variables: a `@media (prefers-color-scheme: dark)` block handles visitors who never touch the theme toggle, while an explicit `:root[data-theme]` override — stamped by a `useResolvedTheme()` hook reacting to the header's Auto/Light/Dark button — wins in both directions once a user makes a choice. The choice persists to `localStorage` so it survives a reload.

Toolkit color coding

Each of the five toolkits (PDF, Image, QR & Barcode, Developer, Health) owns a `--tk-{name}` / `--tk-{name}-soft` / `--tk-{name}-glow` triplet. A single utility class, `.tk-{name}`, remaps those into generic `--tk` / `--tk-soft` / `--tk-glow` variables that any shared component can consume — the mega-menu dot,

the sidebar section heading, the home page toolkit card, the tool badge — all recolor correctly just by having that one class applied to an ancestor element, with zero component-level color logic.

Browser-compatibility constraint: `color-mix()` is banned across the entire codebase. It caused invisible symbol badges on older Safari/Firefox during development; every computed-color use case was replaced with a pre-computed fallback token (e.g. `var(--accent-soft)` instead of `color-mix(in srgb, var(--accent) 12%, transparent)`) — a deliberate trade of a marginally more verbose token file for correctness on a broad, uncontrolled GitHub Pages audience.

Shared component primitives (`@devtools/ui`)

ToolFrame

Panel

Button

CopyButton

DownloadButton

Field

TextArea

TextInput

Select

Checkbox

Note

Empty

KeyValue

StatRow

Spinner

useDebounce (hook)

§8 Tool Catalog — 38 Tools Across 5 Toolkits

Every tool runs entirely client-side; the "runs in your browser" privacy badge shown on each tool page is true by construction — there is no backend to send data to.

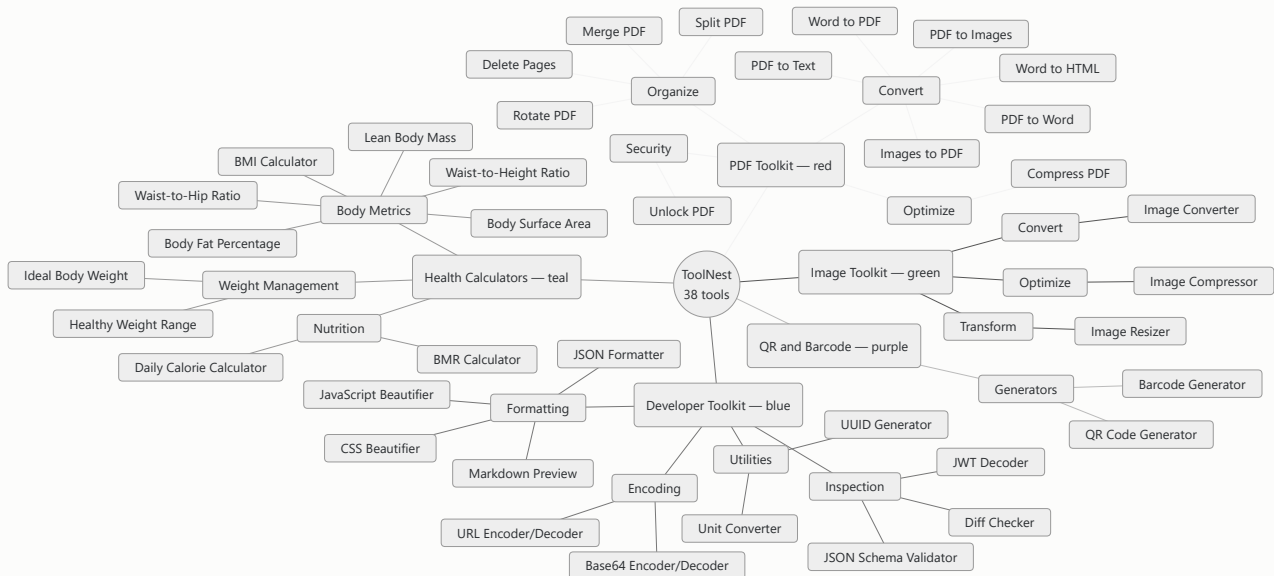


Fig 8.1 — Full tool taxonomy: toolkit → category → tool, matching the mega-menu and sidebar grouping exactly.

PDF Toolkit — 11 tools

Image Toolkit — 3 tools

QR & Barcode — 2 tools

Developer Toolkit — 11 tools

Health Calculators — 10 tools

Each toolkit maps to exactly one Module Federation remote except Health and Developer, which both live inside `utility-tools` alongside QR/Barcode generation — grouped by deployment convenience rather than forced into a one-remote-per-toolkit rule that would have added build overhead without a real isolation benefit for such lightweight, purely computational tools.

§9 Key Libraries & Why They Were Chosen

Portfolio

LIBRARY	VERSION	PURPOSE
next	^12.3.4	Static site generation via Pages Router; pinned for a known-good static export path
react / react-dom	18.2.0	UI runtime
emailjs-com	^3.2.0	Serverless contact form — sends email directly from the browser
react-countup + react-visibility-sensor	^6.3.0 / ^5.1.1	Animated stat counters that trigger only once visible in viewport
swiper	^8.3.0	Touch-friendly carousel for testimonials/partners
wowjs	^1.1.3	Scroll-reveal animation library from the original template

ToolNest — Platform / Infrastructure

LIBRARY	VERSION	PURPOSE
vite	7.3.6	Build tool for the shell and all three remotes — chosen for first-class ESM and fast HMR
@module-federation/vite	^1.20.1	Runtime micro-frontend composition — independent builds, one runtime app
react / react-dom	^19.2.8	UI runtime, shared as a federation singleton across all four apps
react-router-dom	^7.18.2	Client-side routing, also shared as a singleton
@reduxjs/toolkit + react-redux	^2.12.0 / ^9.2.0	Client-only preference state (theme, recents, sidebar)
@tanstack/react-query	^5.101.4	Server-state cache — wired for forward compatibility, currently idle
typescript	^5.7.2	Strict mode + <code>noUncheckedIndexedAccess</code> across the whole workspace

ToolNest — Feature Libraries (all inside `tools-core`)

LIBRARY	POWERS
pdf-lib	Merge, split, rotate, delete pages, unlock — direct PDF byte manipulation
pdfjs-dist	PDF rendering — page-to-image conversion, in-browser preview/thumbnailing

mammoth	Word (.docx) → HTML conversion
docx	Programmatic .docx generation for Word/PDF conversion tools
jspdf + html2canvas	HTML/canvas → PDF export pipeline
qrcode	QR code generation to canvas/SVG
bwip-js	Linear/2D barcode rendering — deliberately excluded from the shared barrel's side effects so its 1.2MB footprint never reaches a tool that doesn't use it
marked + dompurify	Markdown → HTML preview with mandatory sanitization — unsanitized markdown-to-HTML is a stored-XSS vector, called out explicitly in code comments
ajv + ajv-formats	JSON Schema validation tool
js-beautify	JavaScript/CSS/JSON pretty-printing
diff	Text diffing for the Diff Checker tool

§10 Cross-Cutting Design Patterns

1. Single Source of Truth for Data / Routing

Both apps independently arrive at the same pattern: `constants.js` in the portfolio, `routes.ts` in ToolNest. One array feeds every consumer — navigation, prerendering, sitemap generation — so the surfaces cannot drift out of sync with each other.

Why: the classic failure mode this prevents is a nav menu and a mobile menu (or a router and a sitemap) each hand-maintaining their own copy of the same list, which inevitably diverges after the third or fourth edit.

2. Micro-Frontend Composition via Module Federation

Four independently built Vite applications compose into one runtime app via federated module loading, not a build-time monorepo bundle.

Why: proves independent deployability structurally, not just as a claim — the PDF remote could ship a fix without rebuilding the shell or the other two remotes.

3. Narrow, Versioned Cross-Boundary Contracts

`shell-contract` exposes exactly three capabilities (`navigate`, `toast`, `theme`) to every remote, with a version number the shell can check at runtime.

Why: every capability added to a shared interface becomes a coupling point that all consumers depend on simultaneously — keeping it minimal keeps the blast radius of a breaking change small.

4. Contained Failure via Scoped Error Boundaries

Each lazily-loaded remote route is wrapped in its own `ErrorBoundary`, keyed by route so navigation clears a stuck error automatically.

Why: without this, the entire value proposition of splitting into micro-frontends — that one remote's failure can't sink the whole app — would be lost; a single boundary at the app root would blank everything on any remote's crash.

5. Pure-Function Core, Framework at the Edges

`tools-core` contains zero React imports — every PDF, image, health-math, and encoding function is a plain, synchronously testable function operating on primitives and typed arrays.

Why: business logic that doesn't import React can be unit tested without a DOM, reused if a tool ever needs a server-side implementation, and tree-shaken independently per feature — explicitly enabled by marking the package

`sideEffects: false`.

6. Static Prerendering for SPA SEO on a Rewrite-less Host

GitHub Pages has no server-side rewrite rules, so a direct hit on `/tools/jwt-decoder` would 404 without a real file at that path. `postbuild.mjs` writes a per-route `index.html` — a copy of the SPA shell with tool-specific `<title>`, meta description, canonical URL, and JSON-LD stamped in — for all 38 non-hidden routes.

Why: crawlers and shared links get correct per-page metadata on first byte with no redirect flash, while the app still behaves as a client-side SPA for real navigation. A regression test (`postbuild.test.mjs`) locks the regex-based rewrite logic in place, since a silently-broken match would still build successfully but ship every page with the shell's generic metadata.

7. Shared-Dependency Singletons with Loud Failure

React, ReactDOM, and react-router-dom are declared `{ singleton: true, strictVersion: true }` in every federation config.

Why: without this, a version mismatch between the shell and a remote produces "Invalid hook call" deep inside a component at an unpredictable moment. With it, the failure surfaces immediately and clearly at module-load time.

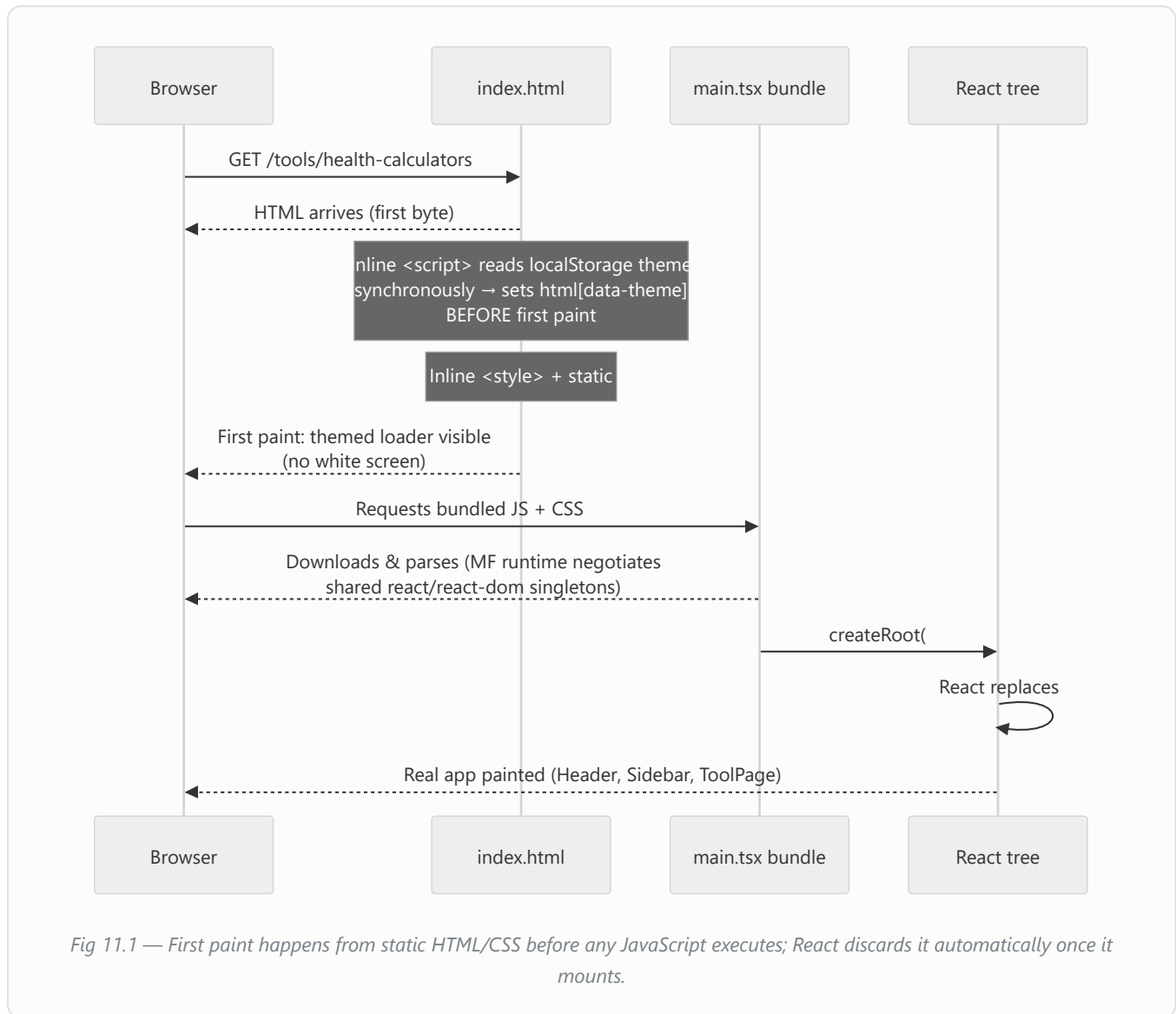
8. Toolkit Color System via One CSS Utility Class

Five toolkits, each with a three-token color triplet, remapped through a single `.tk-{name}` class into generic `--tk` variables that every shared component already knows how to consume.

Why: adding a sixth toolkit later means defining three new CSS variables and one class — zero changes to any component that renders toolkit-colored UI.

§11 Performance Engineering — The Preboot Loader

A pure client-rendered SPA has one structural weakness: the browser has nothing to paint until the JS bundle downloads, parses, and React mounts. On a first visit — no cached bundle, a cold Module Federation runtime negotiating shared singletons — that gap was visible as a blank white screen.



The fix required no framework and no client-side routing changes: a themed spinner and brand mark are written directly into `index.html`, styled by an inline `<style>` block (so it never waits on the external stylesheet bundled with the JS), with a tiny synchronous inline `<script>` that reads the saved theme preference from `localStorage` before first paint — avoiding a flash of the wrong color palette. Because `main.tsx` calls `createRoot(container).render(...)` rather than `hydrateRoot`, React's first commit simply replaces every child of `#root`, so the static loader disappears automatically with no explicit teardown code required.

Engineering takeaway: the highest-leverage fix for a perceived-performance problem was zero JavaScript, applied at exactly the layer (raw HTML) that loads before any framework can possibly help — and it required understanding *why* `createRoot` versus `hydrateRoot` makes the cleanup free.

§12 CI/CD & Deployment Pipeline

One GitHub Actions workflow, two Node.js versions, one strict ordering constraint, and a build/deploy split that makes every branch and pull request provably safe to run.

git push to any branch / PR

build job

Checkout repo

Setup Node 18

npm ci (root)

npm run export
next build && next export
→ out/

Setup Node 24

npm ci --prefix tools



npm run typecheck --prefix
tools

npm test --prefix tools
(postbuild regex regression
test)

npm run build --prefix tools
remotes → shell →
assemble+prerender
→ out/tools/

touch out/.nojekyll

upload-pages-artifact

branch == master AND
not a pull_request?

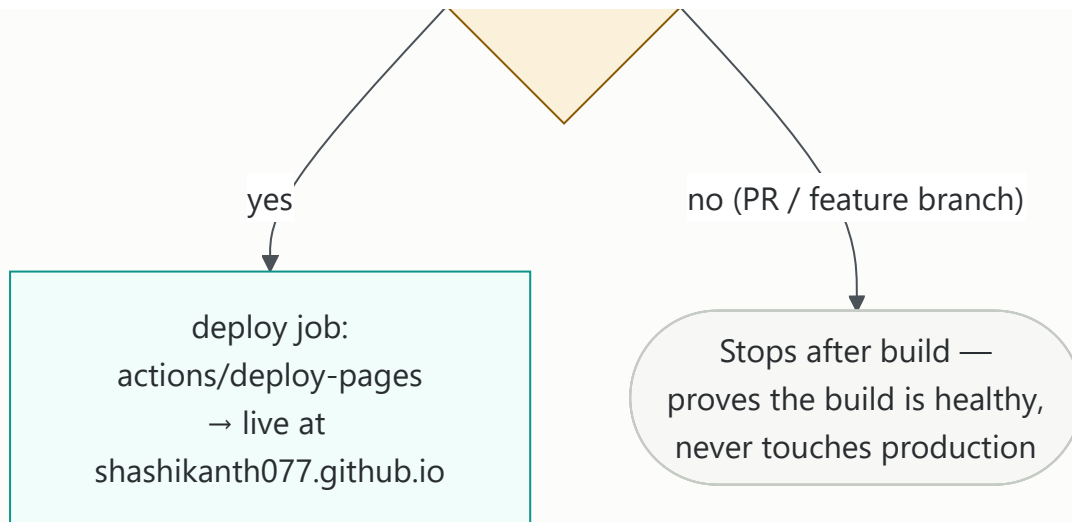


Fig 12.1 — `deploy.yml`: the portfolio's Node-18 export must complete before the tools build writes into the same `out/` directory.

STAGE	DETAIL
Trigger	Every push and pull request run the full <code>build</code> job — a broken build is caught before it reaches <code>master</code>
Publish gate	<code>deploy</code> only runs when <code>github.ref == 'refs/heads/master'</code> and the event is not a pull request — every other ref stops after a successful build, proving health without ever touching production
Node 18	Portfolio only — pinned because the build is known-good and Next.js 12 predates Node 22 by roughly three years
Node 24	ToolNest only — required because <code>@module-federation/vite</code> and Vite 7 refuse to run on Node 18
Ordering constraint	<code>next export</code> must run <i>before</i> the tools build, because it rewrites <code>out/</code> wholesale and would delete <code>out/tools/</code> if run second
Quality gates	<code>tsc</code> strict typecheck, then a regression test on the prerender rewrite logic, both run before the actual build — a passing build with silently-broken metadata is treated as worse than a failed build
Secrets	EmailJS service/template/public keys injected as <code>NEXT_PUBLIC_*</code> env vars at build time from GitHub encrypted secrets — never committed to the repo

§13 Skills Demonstrated — Reviewer's Summary

A condensed index of what this codebase evidences, for anyone assessing it quickly.

Frontend Engineering

React 18 & 19 (hooks-only)

TypeScript strict mode

Next.js SSG / static export

Vite build configuration

CSS custom properties / design tokens

Responsive & dark-mode theming

Accessibility (focus-visible, aria-live, prefers-reduced-motion)

Cross-browser compatibility triage

Architecture & Systems Design

Micro-frontend architecture

Module Federation (runtime composition)

npm workspaces / monorepo structuring

Interface/contract design (shell-contract)

Error boundary / failure containment design

Client state vs. server state boundaries

Single-source-of-truth data modeling

State & Data

Redux Toolkit

TanStack Query

React Router v7

localStorage persistence with defensive fallbacks

Domain Libraries Integrated

PDF manipulation (pdf-lib, pdfjs-dist, jspdf)

Office document conversion (mammoth, docx)

Canvas rendering (html2canvas, qrcode, bwip-js)

Markdown + XSS-safe sanitization (marked, dompurify)

JSON Schema validation (ajv)

Health/biometric formula implementation

DevOps & Delivery

GitHub Actions multi-stage CI/CD

Dual runtime-version pipeline management

Static SEO prerendering & JSON-LD structured data

Sitemap generation

Zero-cost static hosting architecture (GitHub Pages)

Build-order dependency management

Engineering Judgment

Root-cause debugging (MF CSS delivery gap)

Perceived-performance optimization (preboot loader)

Scoping infrastructure to actual need (idle TanStack Query, documented candidly)

Regression testing for silent-failure-prone build logic

Security-conscious defaults (mandatory HTML sanitization)